

Tu semestre, en orden.

Aplicación web para organizar entregas académicas con React, Express y MongoDB.

Documento de evidencias

Proyecto: Entrega

Actividad: Examen Global

Estudiante: nombre pendiente de proporcionar

Asignatura: nombre pendiente de proporcionar

Fecha de elaboración: 23 de septiembre de 2026

Estado: implementación local funcional y probada. PDF y grabación preparados. La ejecución de Docker no pudo comprobarse porque el equipo no tiene Docker instalado. La publicación del nuevo proyecto en GitHub requiere autorización del usuario.

Este documento separa las pruebas ejecutadas de los pasos pendientes. Las capturas y respuestas presentadas provienen de la aplicación y base de datos reales.

Un problema concreto

Problema y contexto

Los estudiantes administran trabajos entre chats, libretas y plataformas. Esto dificulta conocer qué falta, cuál es la fecha límite y qué conviene priorizar.

Objetivo y usuarios

Ofrecer a un estudiante una agenda personal para registrar entregas, consultar fechas y visualizar el avance del semestre desde computadora o teléfono.

Información y funcionalidades

Materias y entregas; título, descripción, fecha, prioridad, estado y tiempo estimado. CRUD completo, búsqueda, filtros, confirmación de borrado, estados de carga, vacío y error.

Alcance

Uso personal local. Sin cuentas, calificaciones, adjuntos o notificaciones. Los datos principales proceden de la API y se almacenan en MongoDB. No se debe exponer como servicio multiusuario sin autenticación y autorización.

Requisitos principales

React y Vite en frontend; Node y Express en backend; API REST; base de datos real; validaciones; Dockerfiles y Compose; volumen persistente; Git progresivo; documentación, capturas y video.

Cada capa tiene su función

```
Usuario / navegador
  | HTTP
React + Vite / Nginx
  | /api/entregas
Express + validaciones + controladores
  | Mongoose
MongoDB / WiredTiger
```

En desarrollo, Vite dirige /api a Express. En Docker, Nginx sirve la compilación y realiza el proxy a backend:5001; el backend se conecta a db:27017. Compose usa DNS interno y espera healthchecks sanos.

Modelo básico de datos

Materia: ObjectId, nombre de hasta 60 caracteres, clave normalizada única y color hexadecimal.

Entrega: ObjectId, título, descripción, materia referenciada, fecha ISO de calendario, prioridad, estado y minutos enteros. Ambos modelos incluyen marcas de creación y actualización.

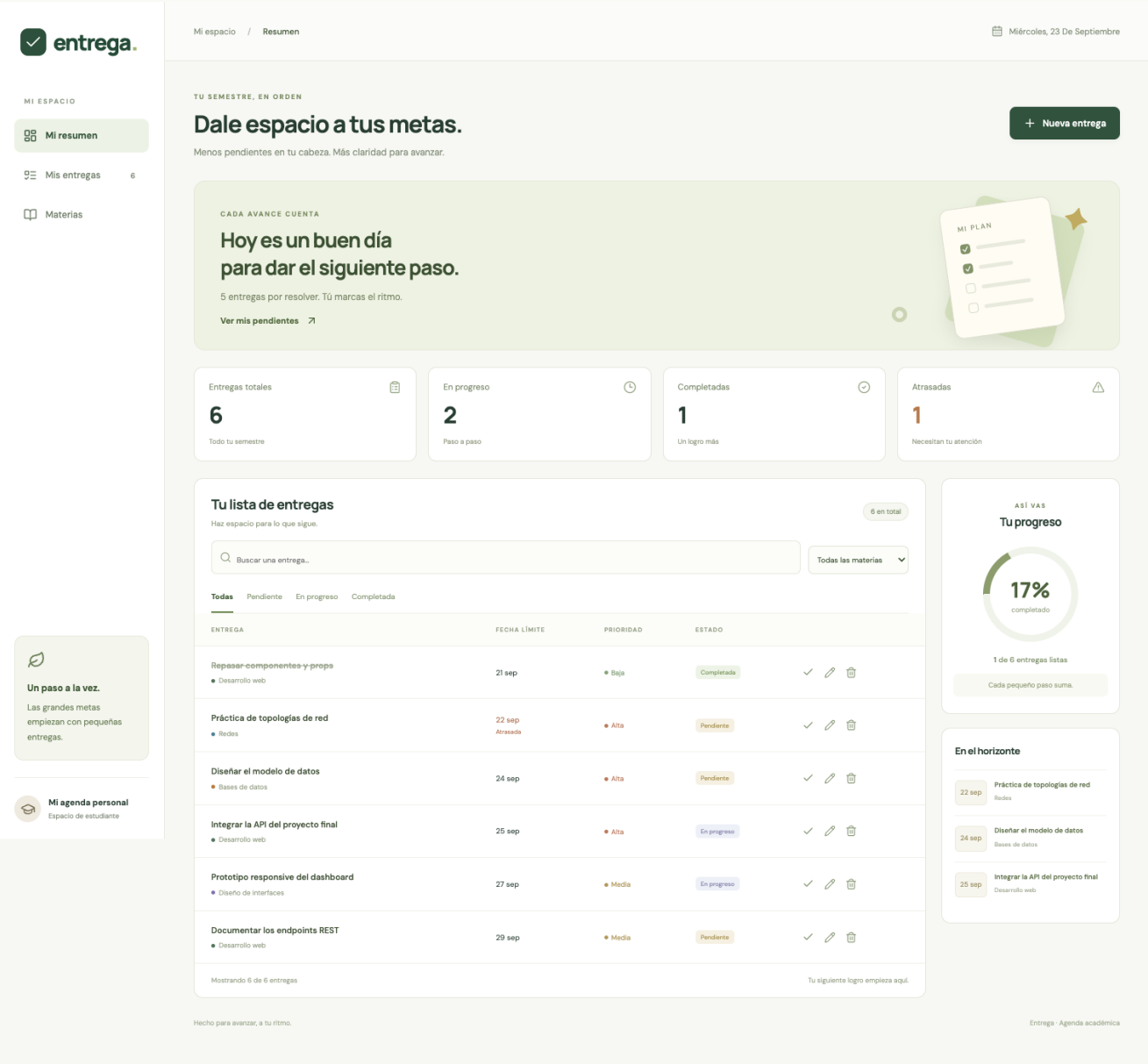
Relación 1:N: una materia tiene muchas entregas. El índice único materia + clave evita títulos duplicados en una asignatura. Una referencia a materia inexistente se rechaza. Los tiempos deben estar entre 5 y 2400 minutos.

Responsabilidades

Componentes de React: formulario, tabla, modal e indicadores. services/api.js centraliza HTTP. Los controladores procesan CRUD, validation.js aplica reglas y los modelos definen la persistencia. Ningún componente concentra toda la aplicación.

Vista principal conectada

Panel con seis registros de demostración cargados en MongoDB. Los indicadores, prioridades, estados y próximas entregas se calculan desde la respuesta de la API.



Captura real del navegador. Los datos de demostración se crean con un script opcional; la aplicación también funciona con una base inicialmente vacía.

Crear, consultar y modificar

El formulario envía POST o PUT. React ofrece controles y límites; Express vuelve a validar tipos, rangos, referencias, fechas y duplicados antes de guardar. La vista detalle consulta GET /api/entregas/:id.

TU AGENDA ACADÉMICA

×

Una nueva meta

Título de la entrega

Documentar el examen global

Materia

Bases de datos

Fecha límite

23/09/2026

Prioridad

Media

Estado

Pendiente

Tiempo estimado (minutos)

60

Descripción (opcional)

Preparar evidencias y explicar la arquitectura del proyecto.

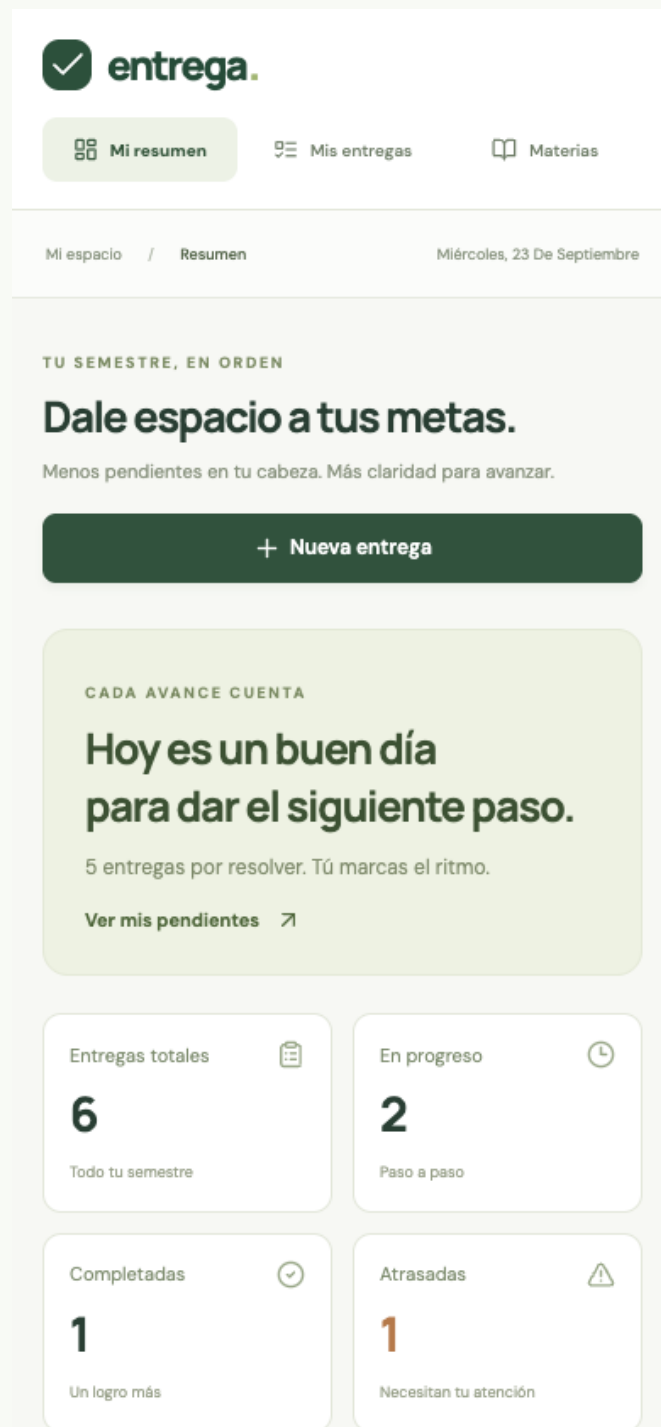
Cancelar

Guardar entrega

Prueba ejecutada: crear una entrega desde React, recargar para comprobar persistencia, abrir su detalle, cambiar estado y tiempo, completar y filtrar. Después se canceló un borrado y se confirmó otro.

Agenda en pantalla móvil

Captura a 390 px. La tabla tiene desplazamiento interno para conservar sus columnas. La página completa no presenta desbordamiento horizontal.



También se comprobó el estado de error de conexión y la recuperación con Reintentar. Los diálogos nativos admiten teclado y Escape; las animaciones respetan movimiento reducido.

Respuestas HTTP verificadas

Evidencia registrada contra Express conectado a MongoDB. El script crea un registro temporal, lo consulta, actualiza y elimina. El último GET confirma que ya no existe.

```
GET /api/health -> HTTP 200
```

```
GET /api/materias -> HTTP 200
```

```
POST /api/entregas -> HTTP 201
```

```
GET /api/entregas/:id -> HTTP 200
```

```
PUT /api/entregas/:id -> HTTP 200
```

```
DELETE /api/entregas/:id -> HTTP 204
```

```
GET /api/entregas/:id -> HTTP 404
```

Archivo completo: docs/evidencia-api.json. Incluye identificadores reales y cuerpos de respuesta. GET /api/health confirmó database: connected.

MongoDB real, no un arreglo

Consulta directa a la base **entrega**: 6 entregas persistidas. El archivo docs/evidencia-db.json incluye colecciones y documentos consultados mediante el driver de MongoDB.

```
{
  "_id": "6ab4877e080e948d6480fea7",
  "titulo": "Integrar la API del proyecto final",
  "materia": "6ab4877e080e948d6480fe9f",
  "fechaLimite": "2026-09-25",
  "estado": "en_progreso",
  "minutos": 120
}
```

Ocho pruebas de integración aprobadas

1. Salud y conexión real.
2. Materias y duplicados.
3. Creación, listado y consulta individual.
4. Tipos, fechas, rangos, referencias y errores.
5. Actualización almacenada.
6. Persistencia tras detener y reabrir mongod.
7. Eliminación y respuesta 404 posterior.
8. JSON incorrecto y rutas inexistentes.

Resultado: 8 aprobadas, 0 fallos. Se detuvo el proceso de base de datos y se reabrieron los mismos archivos WiredTiger. Esto comprueba persistencia local, no constituye todavía evidencia de volumen Docker.

Pruebas adicionales de Chrome: CRUD, recarga, filtros, confirmación, móvil, búsqueda vacía y recuperación ante fallo de red. Compilación Vite correcta.

Docker: implementación y límite

Pendiente de ejecutar: este equipo no dispone de Docker Engine ni Docker Desktop. No se han generado capturas de contenedores ejecutándose ni se afirma que docker compose up --build haya sido probado.

Configuración incluida

Frontend: construcción multietapa con Node y servicio Nginx.

Backend: Node 22, dependencias de producción, usuario sin privilegios.

Base: imagen mongo:7.0 con volumen mongo_data.

Compose: dependencias por salud, red interna y puertos locales configurables.

```
docker compose up --build -d --wait
docker compose ps
docker compose exec backend node scripts/seed.js

# Prueba de persistencia con down y up
bash scripts/verificar-docker.sh
```

El script comprueba HTTP a través del frontend, crea un registro, recrea los contenedores sin borrar el volumen, vuelve a consultar el registro y lo elimina. Guardará evidencia real en docs/docker-verificacion.txt cuando se ejecute.

Instrucciones Dockerfile

FROM: imagen base. **WORKDIR:** directorio de trabajo. **COPY:** incorpora archivos. **RUN:** ejecuta pasos de construcción. **EXPOSE:** documenta un puerto. **CMD:** comando de arranque. Los puertos publicados se definen en Compose.

Lo comprobado y lo pendiente

Resultado

Entrega resuelve una necesidad concreta con un flujo funcional Usuario -> React -> HTTP -> Express -> MongoDB. Se comprobó que crear, leer, modificar y eliminar cambia realmente la base; las validaciones evitan registros inconsistentes y la interfaz comunica errores.

Artefactos

Código y commits progresivos en Examen Global. README con instalación, modelo, API, puertos y entorno. Capturas en docs/capturas. Respuestas API y base de datos en JSON. Pruebas reproducibles. Este PDF y video técnico local en output/. Guion para defensa oral de menos de diez minutos.

Pendientes antes de la entrega final

1. Instalar/iniciar Docker y ejecutar scripts/verificar-docker.sh.
2. Anexar evidencia real de construcción, servicios sanos y volumen.
3. Completar la sección Docker y la defensa oral del video.
4. Proporcionar nombre y materia para personalizar la portada.
5. Autorizar la publicación del nuevo proyecto en un repositorio GitHub.

GitHub: publicación pendiente. Destino propuesto: repositorio Cuh51229458-lang/Ultima-practica, rama examen-global. La revisión automática de permisos bloqueó ese envío por falta de autorización específica; no se presenta ese enlace como proyecto publicado.

Conclusión: la aplicación local y su persistencia quedaron verificadas. La evaluación completa de la contenerización debe cerrarse con pruebas en un entorno Docker disponible.